Data Structures
Stacks



Thareendhra Keerthi Wijayasiriwardhane

Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk





Learning Outcomes

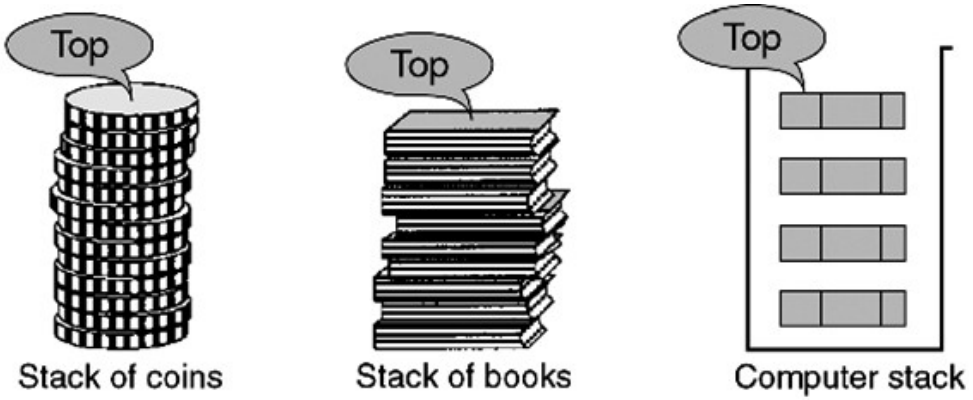
- On completion of this session, you should be able to
  - Define stack data structure
  - Identify the access rule, operations and error conditions associated with a stack
  - Declare array-based and linked list-based stacks
  - Implement array-based and linked list-based stacks
  - Use stacks to solve various real-world problems

Stack

- A linear data structure
- All data additions (push) and removals (pop) are restricted to one end (top)
  - Last In First Out (LIFO)
- Error conditions
  - Overflow

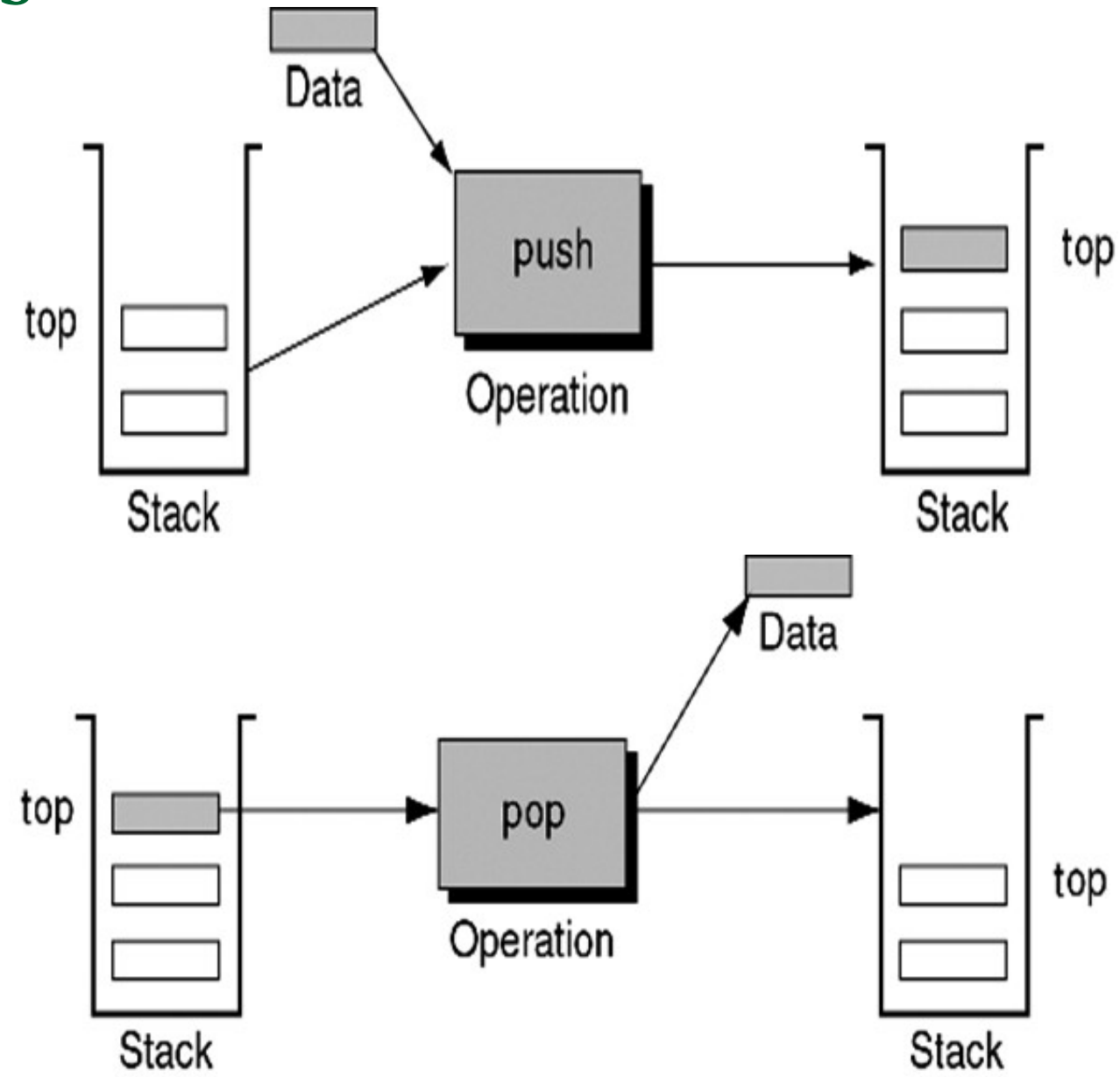
    Happens when trying to push an item onto a full stack
  - Underflow

    Happens when attempting to pop an item from an empty stack



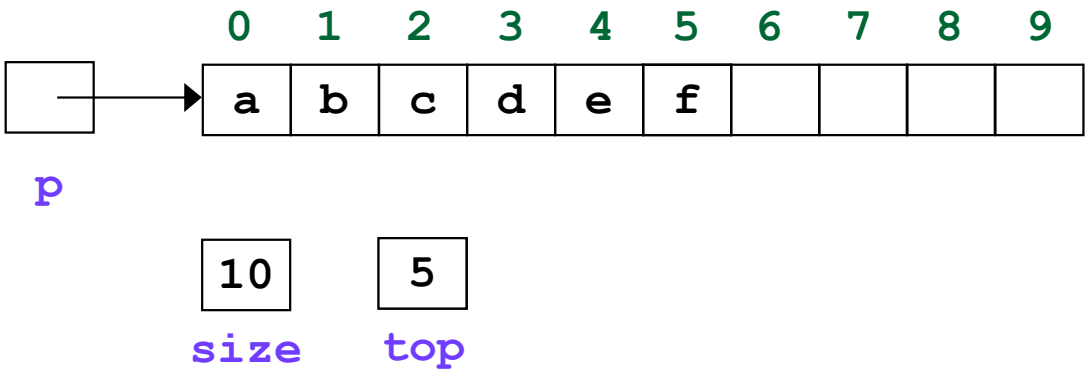
Operations

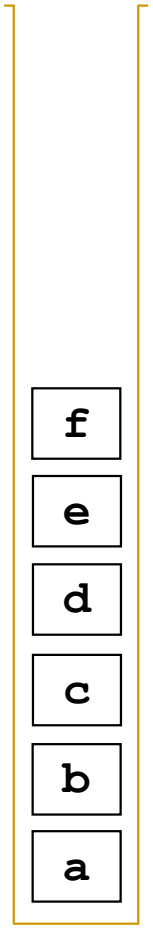
- push( )
- pop( )



Array-based Stack

- Conceptual Stack
- Physical Stack
  - After pushed an item
  - After popped an item





Array-based Stack

- create Operation

```
global variables
p POINTER to DATATYPE, size INTEGER, top INTEGER

ALGORITHM create(VAL s INTEGER)
1  size ← s
2  top ← -1
3  p ← ALLOCATE MEMORY DATATYPE[size]
END create
```

- destroy Operation

```
ALGORITHM destroy()
1  RELEASE MEMORY[] p
END destroy
```

Array-based Stack

- isFull Operation

```
ALGORITHM isFull()
1  IF (top = size - 1)
2     RETURN true
3  ELSE
4     RETURN false
5  END IF
END isFull
```

- isEmpty Operation

```
ALGORITHM isEmpty()
1  IF (top = - 1)
2     RETURN true
3  ELSE
4     RETURN false
5  END IF
END isEmpty
```

Array-based Stack

- push Operation

```
ALGORITHM push(VAL d DATATYPE)
1  IF (isFull())
2     RETURN false
3  ELSE
4     top ← top + 1
5     p[top] ← d
6     RETURN true
7  END IF
END push
```

Array-based Stack

- pop Operation

```
ALGORITHM pop(REF d DATATYPE)
1  IF (isEmpty())
2     RETURN false
3  ELSE
4     d ← p[top]
5     top ← top - 1
6     RETURN true
7  END IF
END pop
```

Array-based Stack

- test Algorithm

```
ALGORITHM test()
1  create(stackSize)
2  READ(data)
3  LOOP(data ≠ endOfData AND push(data))
4     READ(data)
5  END LOOP
6  LOOP(pop(data))
7     PRINT(data)
8  END LOOP
9  destroy()
END test
```

Array-based Stack Implementation

```
//ArrayStack.h header file
class stack
{
   public:
      stack(int); //create() operation
      ~stack(); //destroy() operation
      int push(char);
      int pop(char &);
   private:
      int isEmpty();
      int isFull();
      int size, top;
      char *p;
};
```

Array-based Stack Implementation

```
stack::stack(int s)
{
   size = s;  top = -1;
   p = new char[size]; //allocates memory
}
stack::~stack()
{
   delete[] p; //releases memory
}
int stack::isFull()
{
   return (top == size - 1) ? 1 : 0;
}
int stack::isEmpty()
{
   return (top == -1) ? 1 : 0;
}
```

Array-based Stack Implementation

```
int stack::push(char d)
{
   if (isFull())
      return 0;
   else
   {
      p[++top] = d;
      return 1;
   }
}
```

```
int stack::pop(char &d)
{
   if (isEmpty())
      return 0;
   else
   {
      d = p[top--];
      return 1;
   }
}
```

Array-based Stack Implementation

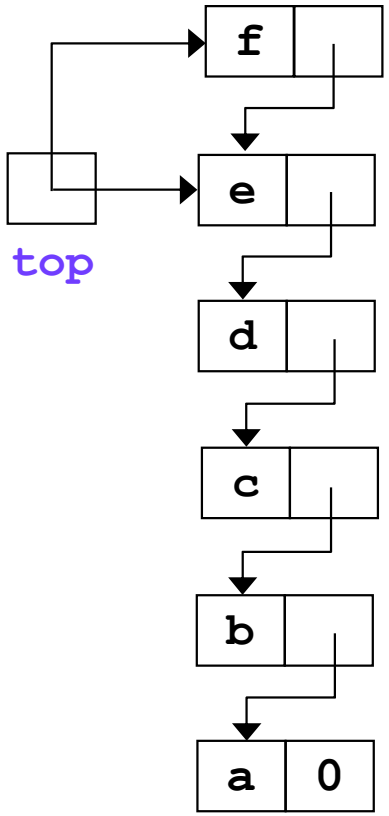
```
//TestArrayStack.cpp source file
#include <iostream>
#include "ArrayStack.h"
int main()
{
   const int stackSize = 10;
   stack s(stackSize);
   char data;
   std::cin >> data;
   while(data != '.' && s.push(data))
      std::cin >> data;
   std::cout << std::endl;
   while(s.pop(data))
      std::cout << data;
   return 0;
}
```

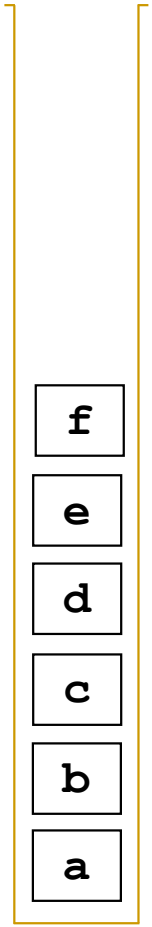
Array-based Stack

- An array-based stack requires a predefined maximum capacity, even if the entire capacity is not used
  - With static arrays, capacity is determined at compile time
  - With dynamic arrays, memory is allocated at run time, but capacity is still fixed after allocation
- Wastes memory in most cases
  - Can be avoided with linked lists that can grow and shrink dynamically during run time

Linked List-based Stack

- Conceptual Stack
- Physical Stack
  - After pushed an item
  - After popped an item





Linked List-based Stack

- create Operation

```
TYPE node
    data DATATYPE
    next POINTER to node
END node
global variables
top POINTER to node

ALGORITHM create()
1  top ← null
END create
```

Linked List-based Stack

- destroy Operation

```
ALGORITHM destroy()
1  LOOP(top ≠ null)
2      p ← top
3      top ← top->next
4      RELEASE MEMORY p
5  END LOOP
END destroy
```

- isEmpty Operation

```
ALGORITHM isEmpty()
1  IF (top = null)
2      RETURN true
3  ELSE
4      RETURN false
5  END IF
END isEmpty
```

Linked List-based Stack

- push Operation

```
ALGORITHM push(VAL d DATATYPE)
1  p ← top
2  top ← ALLOCATE MEMORY node
3  top->data ← d
4  top->next ← p
END push
```

Linked List-based Stack

- pop Operation

```
ALGORITHM pop(REF d DATATYPE)
1  IF (isEmpty())
2     RETURN false
3  ELSE
4     d ← top->data
5     p ← top
6     top ← top->next
7     RELEASE MEMORY p
8     RETURN true
9  END IF
END pop
```

Linked List-based Stack

- test Algorithm

```
ALGORITHM test()
1  create()
2  READ(data)
3  LOOP(data ≠ endOfData)
4     push(data)
5     READ(data)
6  END LOOP
7  LOOP(pop(data))
8     PRINT(data)
9  END LOOP
10 destroy()
END test
```

Linked List-based Stack Implementation

```
//LinkedListStack.h header file
struct node
{
   char data;
   node *next;
};
class stack
{
   public:
      stack(); //create() operation
      ~stack(); //destroy() operation
      void push(char);
      int pop(char &);
   private:
      int isEmpty();
      node *top;
};
```

Linked List-based Stack Implementation

```
stack::stack()
{
   top = 0; //sets to null
}
stack::~stack()
{
   while(top)
   {
      node *p = top;
      top = top->next;
      delete p; //releases memory
   }
}
```

Linked List-based Stack Implementation

```
int stack::isEmpty()
{
   return (top == 0) ? 1 : 0;
}
void stack::push(char d)
{
   node *p = top;
   top = new node; //allocates memory
   top->data = d;
   top->next = p;
}
```

Linked List-based Stack Implementation

```
int stack::pop(char &d)
{
   if (isEmpty())
      return 0;
   else
   {
      d = top->data;
      node *p = top;
      top = top->next;
      delete p;
      return 1;
   }
}
```

Linked List-based Stack Implementation

```
//TestLinkedListStack.cpp source file
#include <iostream>
#include "LinkedListStack.h"
int main()
{
   stack s;
   char data;
   std::cin >> data;
   while(data != '.')
   {
      s.push(data);
      std::cin >> data;
   }
   std::cout << std::endl;
   while(s.pop(data))
      std::cout << data;
   return 0;
}
```

Stack Applications

- Reversing data
- Converting numbers
- Parsing data
- Converting mathematical expressions
- Evaluating mathematical expressions
- Backtracking
  - Recommended References
    - *Data Structures: A Pseudo code Approach with C*, Gilberg and Forouzan, Chapter 3, pp. 102 – 125
    - *C++ How to Program*, Deitel and Deitel, Chapter 21, pp.1038 - 1039

Questions